

Java 实验报告

软件学院 李宗杭 2311451

实验题目：实现模拟对战游戏

一、设计思路

- 1、**CanPlay 接口**：定义了参与战斗的角色所必须具备的基本行为，即攻击和防御。所有具体的角色类（如 Master、Warrior 等）都实现了该接口，从而能进行攻击和防御操作。
- 2、**Actor 抽象类**：作为所有具体角色类的基类，封装了角色的通用属性（名字、血量、攻击指数、防御指数）和通用的攻击、防御行为。通过抽象类的方式，强制要求具体角色类实现 CanPlay 接口的方法，同时提供了一些默认的攻击和防御实现。具体的角色类继承自 Actor 类，继承了通用属性和方法，并可以根据自身特点重写 attack 和 defense 方法。
- 3、**Game 抽象类**：作为所有具体游戏类的基类，定义了游戏的基本框架和必须实现的抽象方法。randomActor 方法用于随机生成系统角色，setActor 方法用于根据用户选择生成角色，endPlay 方法用于处理游戏结束时的逻辑。具体的游戏类（如 Game1、Game2 等）继承自 Game 类，实现了这些抽象方法，从而定义了不同游戏的具体规则。
- 4、**Game1 和 Game2 具体游戏类**：实现了不同类型的游戏，每个游戏类都有自己独特的角色（通过内部类实现）和游戏规则。
- 5、**Play 游戏启动类**：负责游戏的启动流程，包括游戏选择、角色选择、行为选择、战斗逻辑执行和游戏结束处理。通过在主类中实例化 Play 类，并调用其 start 方法启动游戏。
- 6、**Main 类**：作为程序的入口点，负责输出角色信息、获取用户输入的对局数，并循环调用 Play 类的 start 方法进行游戏。

二、游戏规则

- 1、**游戏选择：**游戏开始时，玩家需要从两个游戏（Game1 和 Game2）中选择一个进行游戏。

输入“1”选择 Game1，输入“2”选择 Game2，输入其他内容则视为无效选择，需重新输入。

- 2、**角色选择：**

Game1：玩家选择 Game1 后，可从“Master”（法师）和“Warrior”（战士）两个角色中选择一个。法师的属性为血量 100、攻击 150、防御 50；战士的属性为血量 300、攻击 100、防御 50。输入其他内容则视为无效选择，需重新输入。

Game2：玩家选择 Game2 后，可从“Archer”（射手）和“Knight”（骑士）两个角色中选择一个。射手的属性为血量 200、攻击 100、防御 20；骑士的属性为血量 200、攻击 50、防御 50。输入其他内容则视为无效选择，需重新输入。

- 3、**行为选择：**玩家选择角色后，需要选择一个行为，即“attack”（攻击）或“defense”（防御）。输入其他内容则视为无效选择，需重新输入。

- 4、**系统行为：**系统会随机决定选择角色和行为。

- 5、**战斗逻辑：**

不同角色的防守和攻击效果根据其所属游戏的规则和角色类型而定。包括一般攻击、双倍攻击、强化防守、反弹防守、无效防守。

(1) 玩家防守，系统进攻：当玩家选择防守且系统选择进攻时，先执行玩家的防守操作，再执行系统的攻击操作。

(2) 玩家进攻，系统防守：当玩家选择进攻且系统选择防守时，先执行系统的防守操作，再执行玩家的攻击操作。

(3) 玩家进攻，系统进攻：当玩家和系统都选择进攻时，先执行玩家的攻击操作，再执行系统的攻击操作。

(4) 玩家防守，系统防守：当玩家和系统都选择防守时，不执行任何攻击或防守的实际操作，仅输出双方都选择防守的信息。

- 6、**战斗结果判定：**战斗结束后，根据双方角色剩余的血量判定胜负。若一方角色的血量大于另一方，则该角色获胜；若双方角色的血量相等，则判定为平局。程序会输出双方角色的剩余血量和战斗结果。

三、游戏测试输出

角色信息：	血量	攻击	防御
Game1-Master	100	150	50
Game1-Warrior	300	100	50
Game2-Archer	200	100	20
Game2-Knight	200	50	50

请输入对局数

10

第1局：
请选择游戏：1 或 2
3
无效的游戏选择，请重新输入
0
无效的游戏选择，请重新输入
12
无效的游戏选择，请重新输入
1
请选择角色：Master 或 Warrior
Archer
无效的角色选择，请重新输入
Master
请选择行为：attack 或 defense
move
无效的行为选择，请重新输入
attack
法师【玩家】选择进攻
战士【系统】选择防守
战士【系统】无效防守
法师【玩家】进行双倍攻击，对战士【系统】造成250点伤害
法师【玩家】剩余血量：100，战士【系统】剩余血量：50
法师【玩家】获胜！

第2局：
请选择游戏：1 或 2
1
请选择角色：Master 或 Warrior
Master
请选择行为：attack 或 defense
attack
法师【玩家】选择进攻
法师【系统】选择进攻
法师【玩家】进行一般攻击，对法师【系统】造成100点伤害
法师【系统】进行一般攻击，对法师【玩家】造成100点伤害
法师【玩家】剩余血量：0，法师【系统】剩余血量：0
平局
第3局：
请选择游戏：1 或 2
1
请选择角色：Master 或 Warrior
Master
请选择行为：attack 或 defense
defense
法师【玩家】选择防守
法师【系统】选择进攻
法师【玩家】进行反弹防御，对法师【系统】造成100点伤害
法师【系统】攻击失效
法师【系统】进行一般攻击，对法师【玩家】造成100点伤害
法师【玩家】剩余血量：100，法师【系统】剩余血量：0
法师【玩家】获胜！

第4局：
请选择游戏：1 或 2
1
请选择角色：Master 或 Warrior
Warrior
请选择行为：attack 或 defense
attack
战士【玩家】选择进攻
战士【系统】选择防守
战士【系统】进行强化防御，防御系数变为2倍，为100
战士【玩家】进行双倍攻击，对战士【系统】造成100点伤害
战士【玩家】剩余血量：300，战士【系统】剩余血量：200
战士【玩家】获胜！
第5局：
请选择游戏：1 或 2
1
请选择角色：Master 或 Warrior
Warrior
请选择行为：attack 或 defense
defense
战士【玩家】选择防守
战士【系统】选择进攻
战士【玩家】进行强化防御，防御系数变为2倍，为100
战士【系统】进行双倍攻击，对战士【玩家】造成100点伤害
战士【玩家】剩余血量：200，战士【系统】剩余血量：300
战士【系统】获胜！

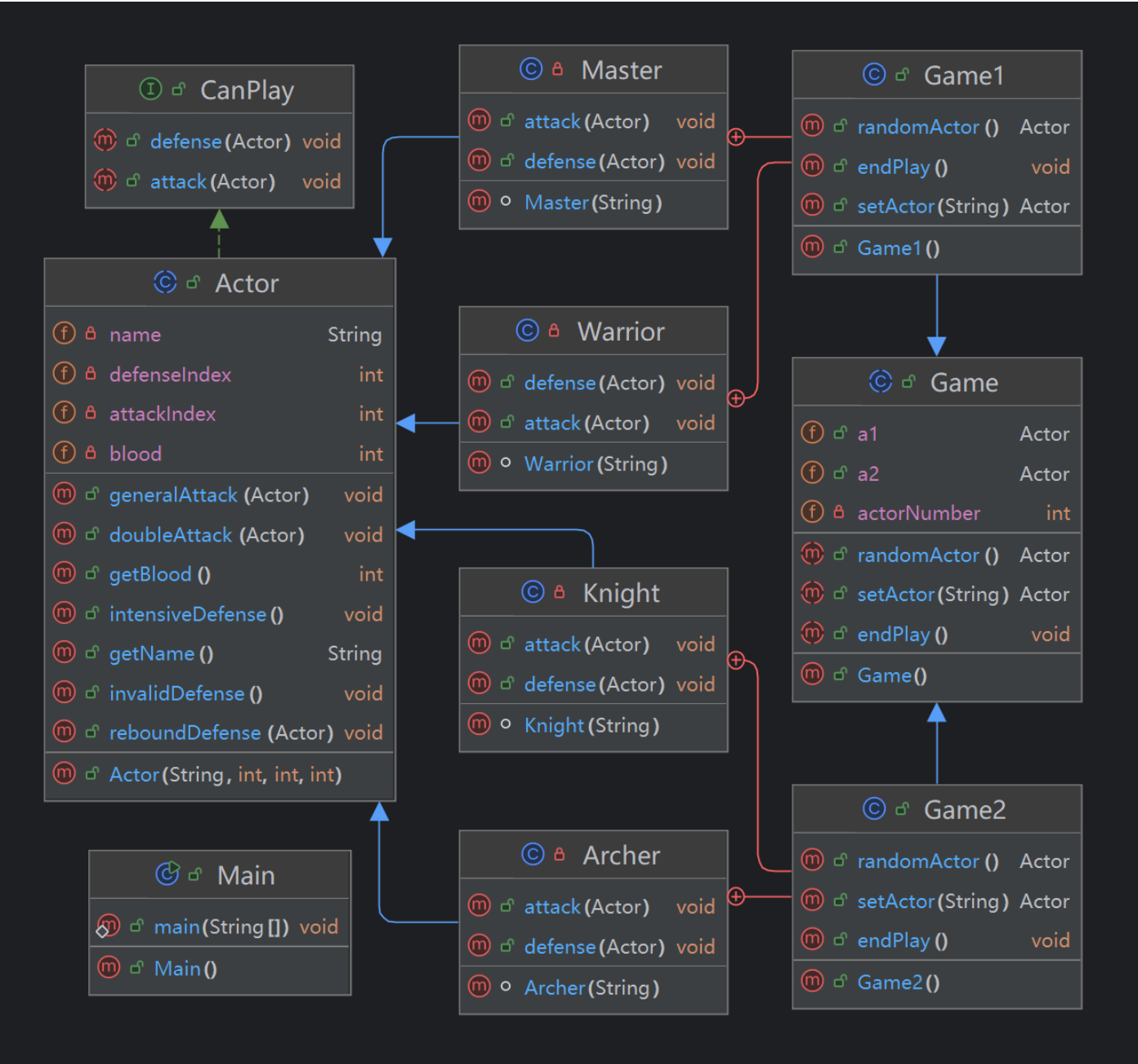
第6局：
请选择游戏：1 或 2
2
请选择角色：Archer 或 Knight
Master
无效的角色选择，请重新输入
Archer
请选择行为：attack 或 defense
fly
无效的行为选择，请重新输入
attack
射手【玩家】选择进攻
射手【系统】选择防守
射手【系统】进行强化防御，防御系数变为2倍，为40
射手【玩家】进行双倍攻击，对射手【系统】造成160点伤害
射手【玩家】剩余血量：200，射手【系统】剩余血量：40
射手【玩家】获胜！

第7局：
请选择游戏：1 或 2
2
请选择角色：Archer 或 Knight
Archer
请选择行为：attack 或 defense
attack
射手【玩家】选择进攻
骑士【系统】选择防守
骑士【系统】无效防守
射手【玩家】进行一般攻击，对骑士【系统】造成50点伤害
射手【玩家】剩余血量：200，骑士【系统】剩余血量：150
射手【玩家】获胜！
第8局：
请选择游戏：1 或 2
2
请选择角色：Archer 或 Knight
Archer
请选择行为：attack 或 defense
defense
射手【玩家】选择防守
射手【系统】选择防守
射手【玩家】剩余血量：200，射手【系统】剩余血量：200
平局

第9局：
请选择游戏：1 或 2
2
请选择角色：Archer 或 Knight
Knight
请选择行为：attack 或 defense
attack
骑士【玩家】选择进攻
骑士【系统】选择进攻
骑士【玩家】进行双倍攻击，对骑士【系统】造成50点伤害
骑士【系统】进行双倍攻击，对骑士【玩家】造成50点伤害
骑士【玩家】剩余血量：150，骑士【系统】剩余血量：150
平局
第10局：
请选择游戏：1 或 2
2
请选择角色：Archer 或 Knight
Knight
请选择行为：attack 或 defense
defense
骑士【玩家】选择防守
骑士【系统】选择防守
骑士【玩家】剩余血量：200，骑士【系统】剩余血量：200
平局

第 1 和第 6 局分别测试两种游戏模式下的输入检查功能， 其余局数测试不同角色和行为的组合功能

四、类图



```
001 package simulated.battle.game;
002
003 public interface CanPlay {
004
005     void attack(Actor a);
006     void defense(Actor a);
007 }
008 package simulated.battle.game;
009
010 public abstract class Actor implements CanPlay {
011
012     private String name;
013     private int blood;
014     private int attackIndex;
015     private int defenseIndex;
016
017     public Actor(String name, int blood, int attackIndex, int defenseIndex) {
018         this.name = name;
019         this.blood = blood;
020         this.attackIndex = attackIndex;
021         this.defenseIndex = defenseIndex;
022     }
023
024     public void generalAttack(Actor a) {
025         int damage = attackIndex - a.defenseIndex;
026         a.blood = a.blood - damage;
027         System.out.println(name + "进行一般攻击, 对" + a.name + "造成" + damage + "点伤害");
028     };
029 }
030
031 public void doubleAttack(Actor a) {
032     int damage = attackIndex * 2 - a.defenseIndex;
033     a.blood = a.blood - damage;
034     System.out.println(name + "进行双倍攻击, 对" + a.name + "造成" + damage + "点伤害");
035 };
036 }
037
038 public void reboundDefense(Actor a) {
039     int damage = attackIndex - a.defenseIndex;
040     a.blood = a.blood - damage;
041     blood = blood + damage;
042     System.out.println(name + "进行反弹防御, 对" + a.name + "造成" + damage + "点伤害");
043 };
044     System.out.println(a.name + "攻击失效");
045 }
046
047 public void intensiveDefense() {
048     defenseIndex *= 2;
049     System.out.println(name + "进行强化防御, 防御系数变为 2 倍, 为" + defenseIndex);
```

```

050     }
051
052     public void invalidDefense() {
053         System.out.println(name + "无效防守");
054     }
055
056     public String getName() {
057         return name;
058     }
059
060     public int getBlood() {
061         return blood;
062     }
063
064 }
065 package simulated.battle.game;
066
067 public abstract class Game {
068
069     private int actorNumber;
070     public Actor a1;
071     public Actor a2;
072
073     public abstract Actor randomActor();
074     public abstract Actor setActor(String type);
075     public abstract void endPlay();
076 }
077 package simulated.battle.game;
078
079 public class Game1 extends Game{
080
081     private class Master extends Actor{
082
083         Master(String name){
084             super(name,100,150,50);
085         }
086
087         @Override
088         public void attack(Actor a){
089             if (a instanceof Master){
090                 generalAttack(a);
091             }else {
092                 doubleAttack(a);
093             }
094         }
095
096         @Override
097         public void defense(Actor a){
098             if (a instanceof Master){

```

```
099         reboundDefense(a);
100     }else {
101         invalidDefense();
102     }
103 }
104 }
105
106 private class Warrior extends Actor{
107
108     Warrior(String name){
109         super(name,300,100,50);
110     }
111
112     @Override
113     public void attack(Actor a){
114         if (a instanceof Warrior){
115             doubleAttack(a);
116         }else {
117             generalAttack(a);
118         }
119     }
120
121     @Override
122     public void defense(Actor a){
123         if (a instanceof Warrior){
124             intensiveDefense();
125         }else {
126             invalidDefense();
127         }
128     }
129 }
130
131 @Override
132 public Actor randomActor() {
133     if(Math.random()<0.5){
134         return new Master("法师[系统]");
135     } else {
136         return new Warrior("战士[系统]");
137     }
138 }
139
140 @Override
141 public Actor setActor(String type) {
142     if(type.equals("Master")){
143         return new Master("法师[玩家]");
144     }else {
145         return new Warrior("战士[玩家]");
146     }
147 }
```

```

148
149 @Override
150 public void endPlay(){
151     System.out.print(a1.getName()+"剩余血量: "+a1.getBlood()+" ", );
152     System.out.println(a2.getName()+"剩余血量: "+a2.getBlood());
153     if(a1.getBlood()>a2.getBlood()){
154         System.out.println(a1.getName()+"获胜! ");
155     }else if(a2.getBlood()>a1.getBlood()){
156         System.out.println(a2.getName()+"获胜! ");
157     }else {
158         System.out.println("平局");
159     }
160 }
161 }
162 package simulated.battle.game;
163
164 public class Game2 extends Game{
165
166     private class Archer extends Actor{
167
168         Archer(String name){
169             super(name,200,100,20);
170         }
171
172         @Override
173         public void attack(Actor a){
174             if (a instanceof Archer){
175                 doubleAttack(a);
176             }else {
177                 generalAttack(a);
178             }
179         }
180
181         @Override
182         public void defense(Actor a){
183             if (a instanceof Archer){
184                 intensiveDefense();
185             }else {
186                 invalidDefense();
187             }
188         }
189     }
190
191     private class Knight extends Actor{
192
193         Knight(String name){
194             super(name,200,50,50);
195         }
196

```

```

197     @Override
198     public void attack(Actor a){
199         if (a instanceof Knight){
200             doubleAttack(a);
201         }else {
202             generalAttack(a);
203         }
204     }
205
206     @Override
207     public void defense(Actor a){
208         if (a instanceof Knight){
209             reboundDefense(a);
210         }else {
211             invalidDefense();
212         }
213     }
214 }
215
216 @Override
217 public Actor randomActor() {
218     if(Math.random()<0.5){
219         return new Archer("射手[系统]");
220     } else {
221         return new Knight("骑士[系统]");
222     }
223 }
224
225 @Override
226 public Actor setActor(String type) {
227     if(type.equals("Archer")){
228         return new Archer("射手[玩家]");
229     }else {
230         return new Knight("骑士[玩家]");
231     }
232 }
233
234 @Override
235 public void endPlay(){
236     System.out.print(a1.getName()+"剩余血量: "+a1.getBlood()+" ");
237     System.out.println(a2.getName()+"剩余血量: "+a2.getBlood());
238     if(a1.getBlood()>a2.getBlood()){
239         System.out.println(a1.getName()+"获胜! ");
240     }else if(a2.getBlood()>a1.getBlood()){
241         System.out.println(a2.getName()+"获胜! ");
242     }else {
243         System.out.println("平局");
244     }
245 }

```

```
246 }
247 package simulated.battle.game;
248
249 import java.util.Scanner;
250
251 public class Play {
252
253     public void start() {
254
255         Scanner scanner = new Scanner(System.in);
256
257         int gameChoice = 0;
258         String input;
259         Game game=null;
260
261         System.out.println("请选择游戏: 1 或 2");
262         while (gameChoice == 0) {
263
264             input = scanner.next();
265             try {
266                 gameChoice = Integer.parseInt(input);
267             } catch (NumberFormatException e) {
268                 System.out.println("无效的游戏选择, 请重新输入");
269                 continue;
270             }
271             if (gameChoice == 1) {
272                 game = new Game1();
273             } else if (gameChoice == 2) {
274                 game = new Game2();
275             } else {
276                 System.out.println("无效的游戏选择, 请重新输入");
277                 gameChoice = 0;
278             }
279         }
280         String roleChoice = null;
281         if (gameChoice == 1) {
282             System.out.println("请选择角色: Master 或 Warrior");
283         } else {
284             System.out.println("请选择角色: Archer 或 Knight");
285         }
286         while (roleChoice == null) {
287             roleChoice = scanner.next();
288             if (gameChoice == 1) {
289                 if (!(roleChoice.equals("Master") || roleChoice.equals("Warrior"))) {
290                     roleChoice = null;
291                 }
292             } else {
293                 if (!(roleChoice.equals("Archer") || roleChoice.equals("Knight"))) {
294                     roleChoice = null;
```

```

295         }
296     }
297     if (roleChoice == null) {
298         System.out.println("无效的角色选择, 请重新输入");
299     }
300 }
301
302 game.a1 = game.setActor(roleChoice);
303 game.a2 = game.randomActor();
304
305 String actionChoice = null;
306 System.out.println("请选择行为: attack 或 defense");
307 while (actionChoice == null) {
308     actionChoice = scanner.next();
309     if (!(actionChoice.equals("attack") || actionChoice.equals("defense"))) {
310         actionChoice = null;
311         System.out.println("无效的行为选择, 请重新输入");
312     }
313 }
314
315 String systemAction;
316 if (Math.random() < 0.5) {
317     systemAction = "attack";
318 } else {
319     systemAction = "defense";
320 }
321
322 if (actionChoice.equals("defense") && systemAction.equals("attack")) {
323     System.out.println(game.a1.getName()+"选择防守");
324     System.out.println(game.a2.getName()+"选择进攻");
325     game.a1.defense(game.a2);
326     game.a2.attack(game.a1);
327 } else if (actionChoice.equals("attack") && systemAction.equals("defense")) {
328     System.out.println(game.a1.getName()+"选择进攻");
329     System.out.println(game.a2.getName()+"选择防守");
330     game.a2.defense(game.a1);
331     game.a1.attack(game.a2);
332 } else if (actionChoice.equals("attack") && systemAction.equals("attack")) {
333     System.out.println(game.a1.getName()+"选择进攻");
334     System.out.println(game.a2.getName()+"选择进攻");
335     game.a1.attack(game.a2);
336     game.a2.attack(game.a1);
337 } else {
338     System.out.println(game.a1.getName()+"选择防守");
339     System.out.println(game.a2.getName()+"选择防守");
340 }
341 game.endPlay();
342 }
343 }

```

```
344 package simulated.battle.game;
345
346 import java.util.Scanner;
347
348 public class Main {
349     public static void main(String[] args) {
350         Scanner scanner = new Scanner(System.in);
351         System.out.println("""
352             角色信息:           血量  攻击  防御
353             Game1-Master      100  150  50
354             Game1-Warrior     300  100  50
355             Game2-Archer      200  100  20
356             Game2-Knight      200   50  50
357             """);
358         System.out.println("请输入对局数");
359         int round = scanner.nextInt();
360         for (int i = 0; i < round; i++) {
361             System.out.println("第" + (i+1) + "局: ");
362             Play play = new Play();
363             play.start();
364         }
365     }
366 }
```

